Foundations of Cybersecurity / Applied Cryptography
17 January 2023

Exercise n. 1 [8]

1. Illustrate the authenticated-encryption (AE) scheme "encrypt and authenticate".
2. Argue whether it can be considered secure.
3. Illustrate the AE scheme "encrypt then authenticate".

Exercise n. 2 [16]

Assume a threat model in which an adversary can steal the password file and perform an off-line Rainbow Table attack.

- Assume we adopt the following salted hashing technique:

$$h = H^{1000}(p) \oplus s$$

whet p is the plaintext password, s is a 128-bit random salt, and H is a secure one-way has function. The pair (h, s) is stored in the password file on disk. Is this approach secure? Explain why.

- Assume now we adopt the following salted hashing technique:

$$h = H(p||s)$$

and again, we store the pair (h, s) in the password file on disk. Is this choice better than the previous one? Explain why.

- With reference to the second hashing scheme, if users employ 8 characters passwords chosen over the lowercase alphanumeric characters, how many bits long should be the salt to prevent an attacker able to (pre-)compute 2^{70} passwords from employing a Rainbow Table attack?
- Does the previous amount of random salt prevent an attacker from brute-forcing a single password?

Exercise n. 3 [6]

A client of yours ask you to perform a black-box penetration test to the login page of his e-commerce website. Such a page displays a form with two fields, a "`Username`" field, and a "`Password`" field. The client provides you the following credentials: "`Engineer`" as username and "`Computer`" as password. You make an attempt to login by specifying "`Engineer' -- `" as username, and a password of your invention. The login fails. Is this test sufficient to conclude that the website is safe against SQL injection attack?

Solution

EXERCISE N.1.

See theory

EXERCISE N.2

• Yes, the chosen salted hash strategy allows an attacker to easily remove the effect of the salt through simply computing $h \oplus s$ and then lead any TMTO attack of his choice.

• The choice is a better one: there is no trivial way to remove the salt from the hash.

• Considering that an 8 characters password over lowercase alphanumeric characters lies in a $(26 + 10)^8 \approx 2^{41}$ wide password space, we need to employ a salt longer than $70 - 41 = 29$ bits to prevent an attacker to mount a TMTO attack precomputing all the possible hashes for all the possible salts.

• No: the cost for the computation of a single password is 2^{41} operations, thus it is feasible to brute-force it.

EXERCISE N.3.

After the test we cannot be not sure that the login system is safe. We need to perform at least one more test. If the webpage contained a query structured as follows

```
SELECT * FROM Users WHERE PwdHash=hash($pwd) and UName='$Username'
```

the login system would not be safe against a tautology attack. As we are performing a black box pentest, we cannot know and thus it is wise to also perform a test in which we specify, for instance, `"Engineer' OR 1=1 --"` as username. An alternative test may be `"Engineer' OR 'a' = 'a"`

Notice that inserting the tautology inside the password field would not work, since most often than not, the content of the password form will be hashed before being evaluated by the query.